

Computing the probability of sentences with language models

Computing the probability of a sentence

Apply the Chain Rule: the trigram model

$$\begin{aligned} P(w_1, \dots, w_n) \\ &\approx P(w_1)P(w_2 \mid w_1)P(w_3 \mid w_1, w_2) \dots P(w_n \mid w_{n-2}, w_{n-1}) \\ &\approx P(w_1)P(w_2 \mid w_1) \prod_{i=3}^n P(w_i \mid w_{i-2}, w_{i-1}) \end{aligned}$$

- Notice that the length of the sentence n is variable
- What is size of the event space (e.g. the total number of possible events/sentences)?

Variable length sequences

Let $V = \{a, b\}$ and the language L be V^*

Consider a unigram model: $P(a) = P(b) = 0.5$

So strings in this language L are:

a	0.5
b	0.5
aa	0.5^2
bb	0.5^2
$\dots$	

The sum over all strings in L should be equal to 1

But $P(a) + P(b) + P(aa) + P(bb) = 1.5!!!$

The **stop** symbol

What went wrong? We need to model variable length sentences

Add an explicit probability for the stop symbol

$$P(a) = P(b) = 0.25 \quad P(\text{stop}) = 0.5$$

Now strings have the following probabilities:

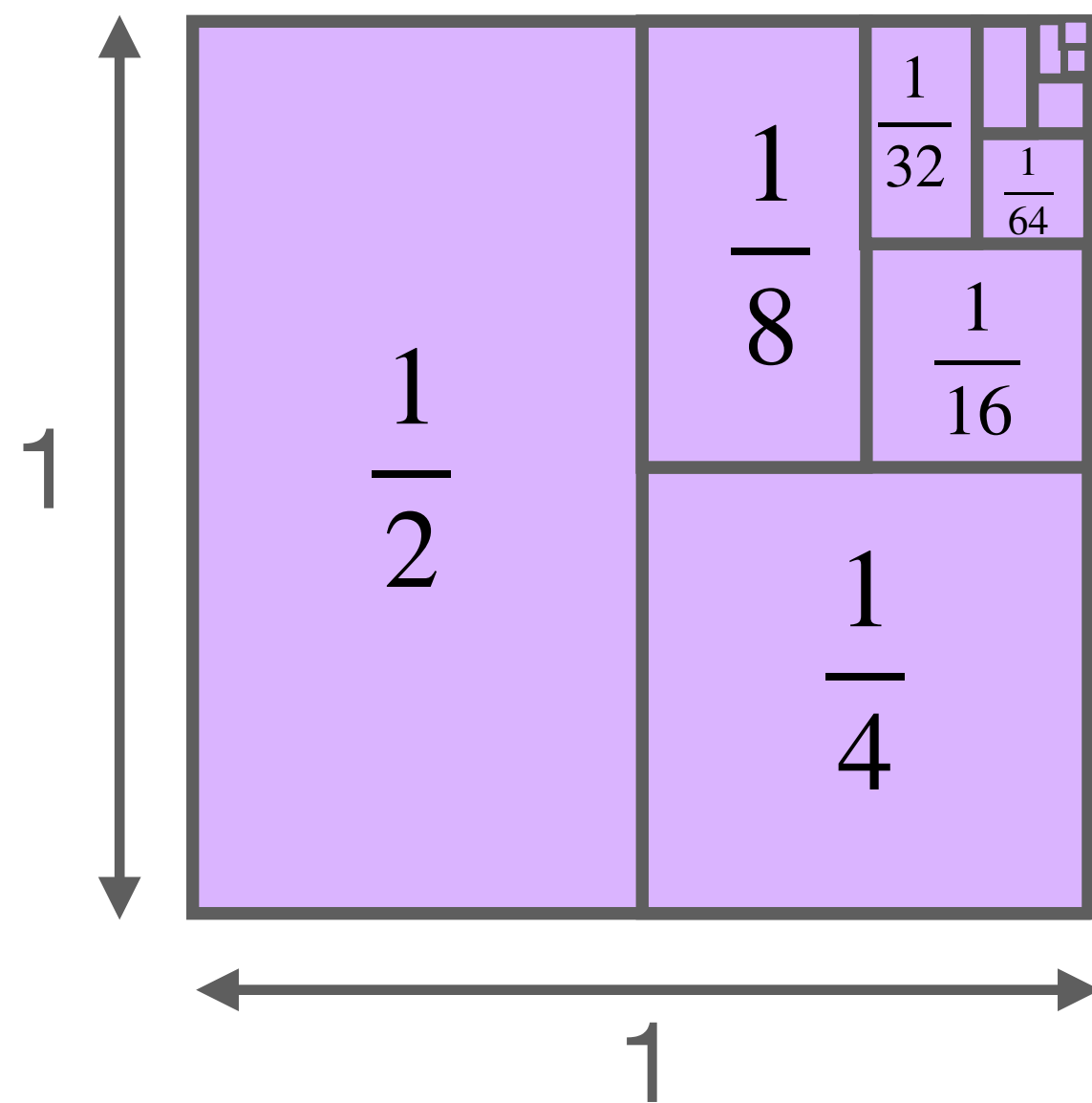
$$\begin{array}{ll} \text{stop} & 0.5 \\ a \text{ stop} & 0.25 \times 0.5 = 0.125 \\ b \text{ stop} & 0.25 \times 0.5 = 0.125 \\ aa \text{ stop} & 0.25^2 \times 0.5 = 0.03125 \\ bb \text{ stop} & 0.25^2 \times 0.5 = 0.03125 \\ & \dots \end{array}$$

The sum is no longer greater than one!

The stop symbol

- ▶ With this new stop symbol we can show that $\sum_{u \in L} P(u) = 1$
 Notice that the probability of any sequence of length n is $0.25^n \times 0.5$
 Also there are 2^n sequences of length n

Geometric Series



$$\sum_{s \in L} P(s) =$$

$$\sum_{n=0}^{\infty} 2^n \times 0.25^n \times 0.5$$

$$\sum_{n=0}^{\infty} 0.5^n \times 0.5 = \sum_{n=0}^{\infty} 0.5^{n+1}$$

$$\sum_{n=1}^{\infty} 0.5^n = 1$$

The stop symbol

- ▶ With this new stop symbol we can show that $\sum_{u \in L} P(u) = 1$.
- ▶ Let $p_s = P(\text{stop})$, the probability of the stop symbol.
- ▶ First, let us show that the probability of all sequences of length n is $p(n) = p_s(1 - p_s)^n$

$$p(n) = \sum_{w_1, \dots, w_n} p(w_1, \dots, w_n) \times p_s \text{ where } w_j \neq \text{stop}$$

$$= p_s \sum_{w_1} \dots \sum_{w_n} p(w_1, \dots, w_n)$$

$$= p_s \sum_{w_1} \dots \sum_{w_n} p(w_1) \dots p(w_n)$$

$$= p_s \sum_{w_1} p(w_1) \dots \sum_{w_n} p(w_n)$$

$$= p_s \prod_{j=1}^n \sum_{w_j} p(w_j)$$

$$= p_s(1 - p_s)^n$$

Note that

$$\sum_{w \neq \text{stop}} P(w) = 1 - p_s$$



More generally

The stop symbol

- ▶ With this new stop symbol we can show that $\sum_{u \in L} P(u) = 1$.
- ▶ Let $p_s = P(\text{stop})$, the probability of the stop symbol.
- ▶ Using that the probability of all sequences of length n is $p(n) = p_s(1 - p_s)^n$

$$\begin{aligned} \sum_{u \in L} P(u) &= \sum_{n=0}^{\infty} p(n) = \sum_{n=0}^{\infty} p_s(1 - p_s)^n \\ &= p_s \sum_{n=0}^{\infty} (1 - p_s)^n \\ &= p_s \frac{1}{1 - (1 - p_s)} = p_s \frac{1}{p_s} = 1 \end{aligned}$$

Summary: Estimating language models

- ▶ Predict probability of sequence of words
- ▶ Need to handle **data sparsity**
use Markov assumption and smoothing
Independence assumptions
Reallocate probability mass
Ensure proper probability
- ▶ Later: Neural language models

Use Chain rule and approximate using a neural network

$$p(w_1, \dots, w_n) \approx \prod_t p(w_{t+1} \mid \underbrace{\phi(w_1, \dots, w_t)}_{\text{capture history with vector } s(t)})$$

How are language models used?

How is it incorporated in applications?

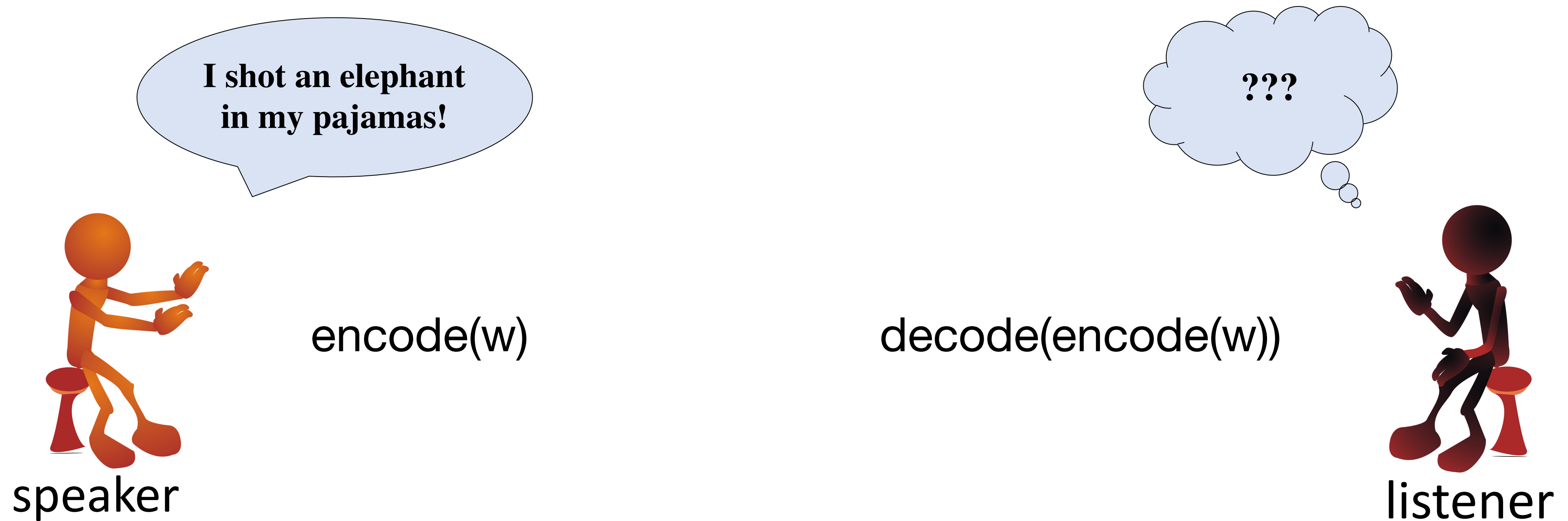
How do we use the language model for generating?

How are language models used?

How is it incorporated in applications?

How do we use the language model for generating?

Automatic Speech Recognition (ASR)

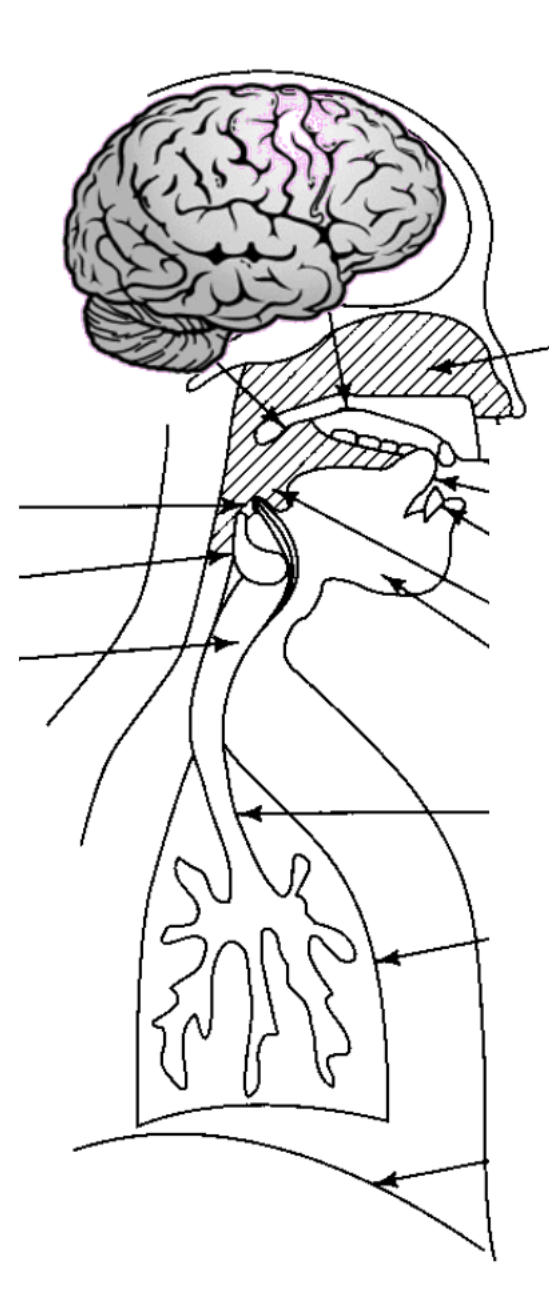


Noisy Channel

Let a be the encoded audio signal.

We want to predict a sentence given audio signal:

$$w^* = \arg \max_w P(w|a)$$



speaker

encode(w)

decode(encode(w))



listener

Noisy Channel

The noisy channel approach

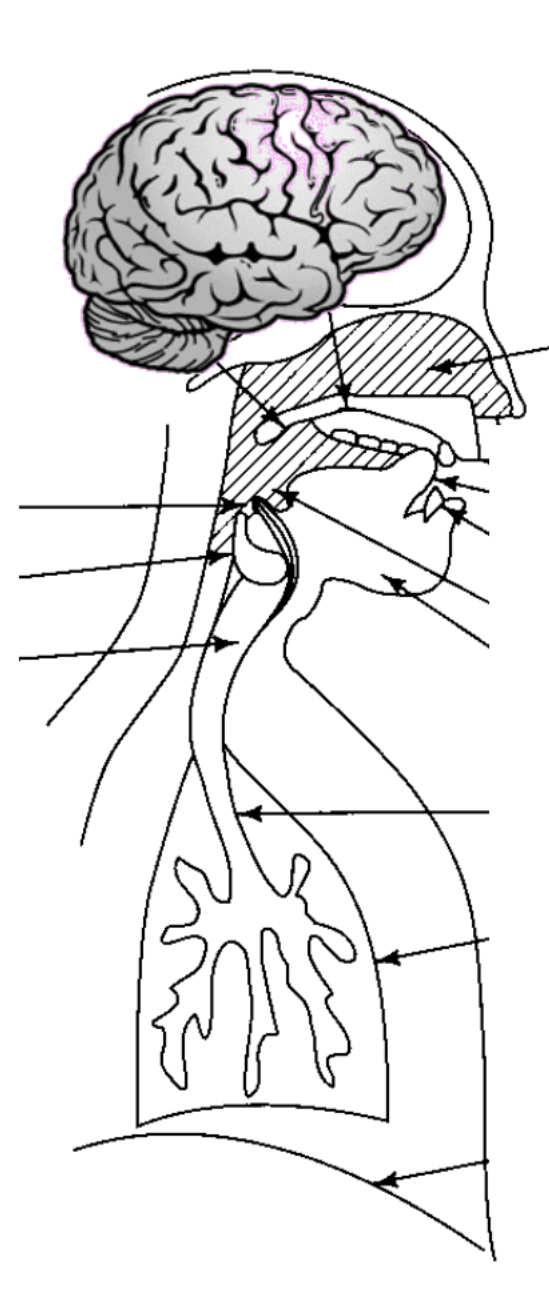
$$\begin{aligned} w^* &= \arg \max_w P(w|a) \\ &= \arg \max_w P(a|w)P(w)/P(a) \\ &\propto \arg \max_w P(a|w)P(w) \end{aligned}$$

Acoustic Model

Language Model

encode(w)

decode(encode(w))

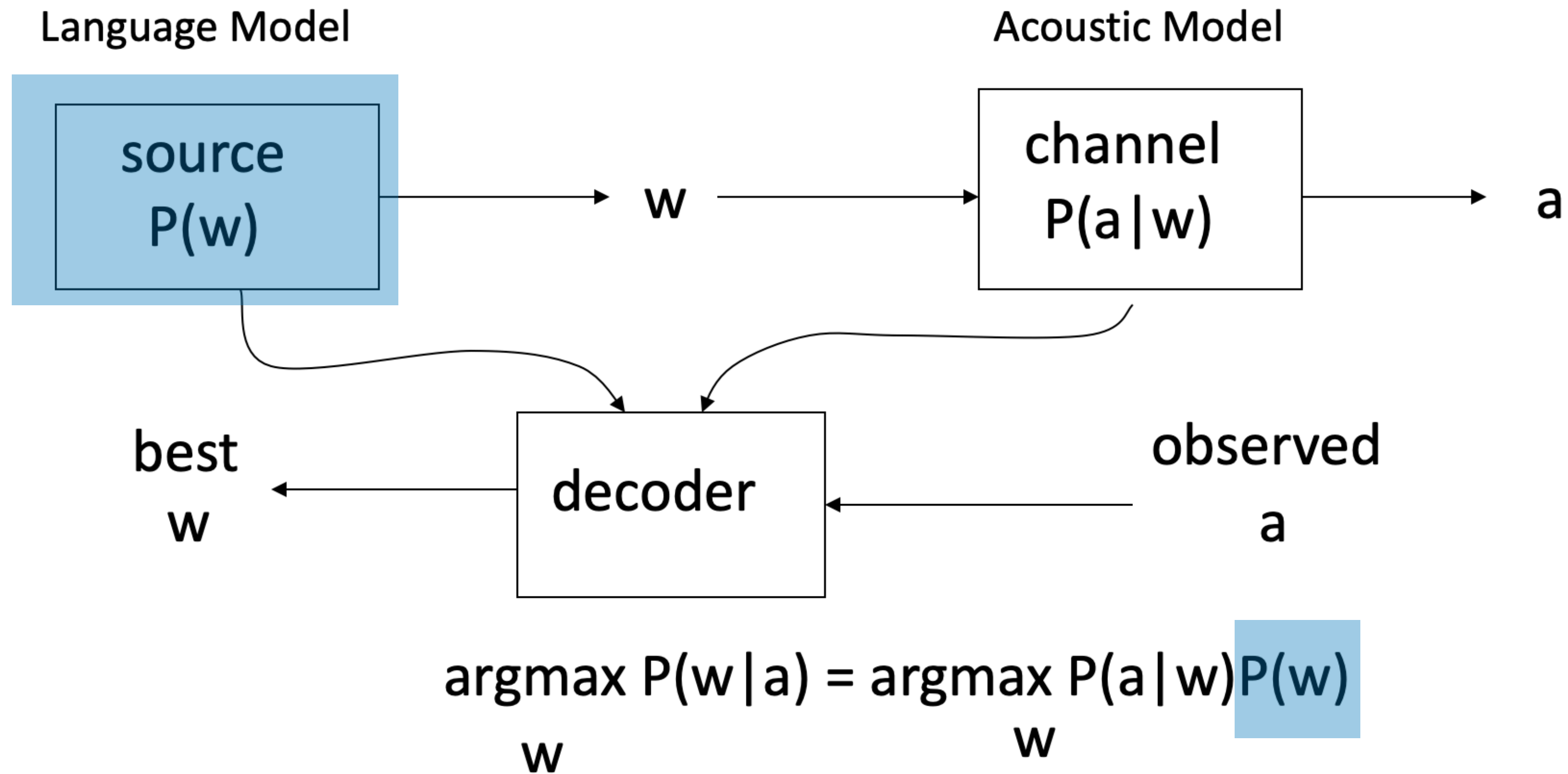


speaker

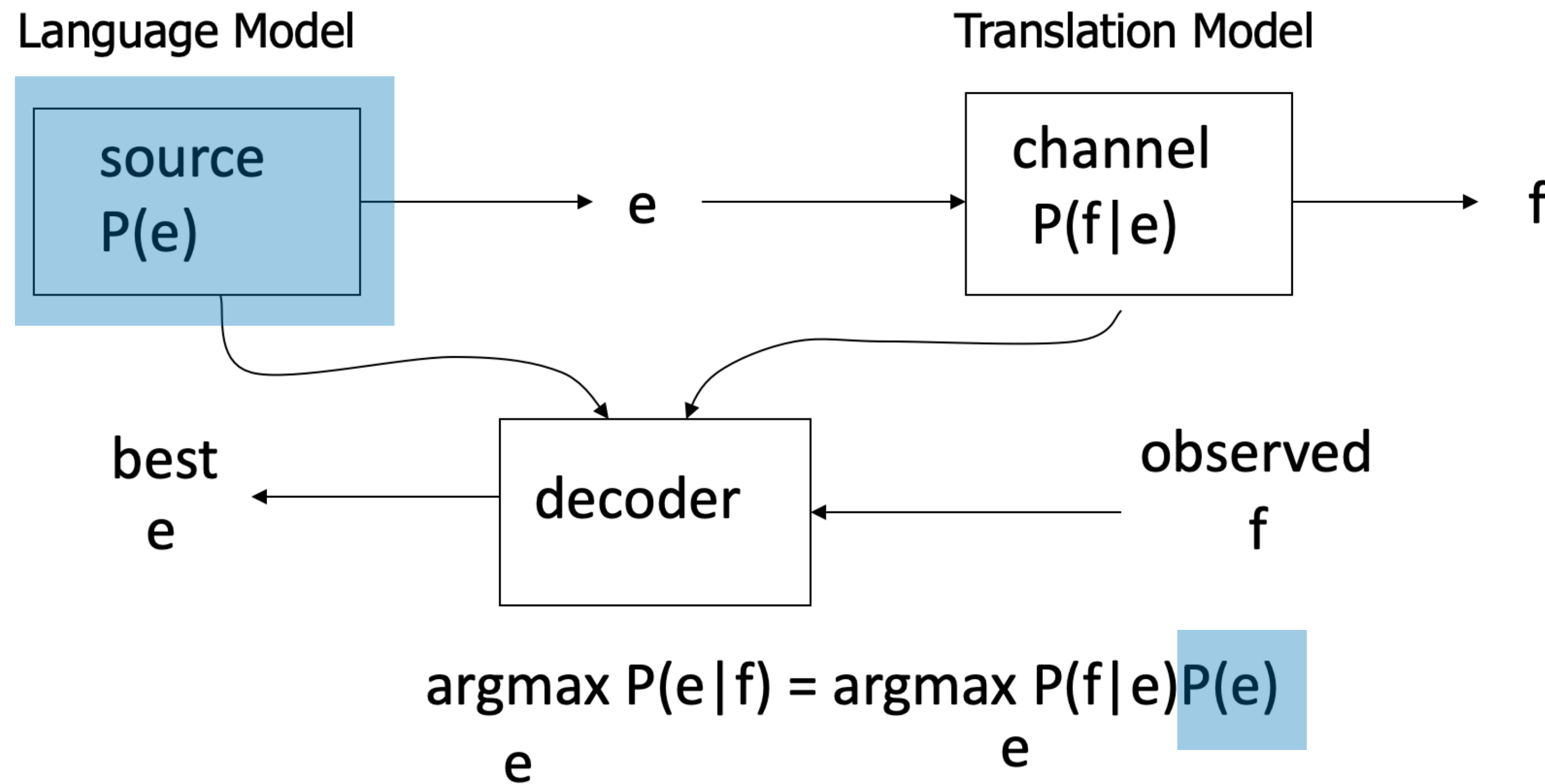


listener

Noisy Channel (Speech Recognition)



Noisy Channel (Machine Translation)

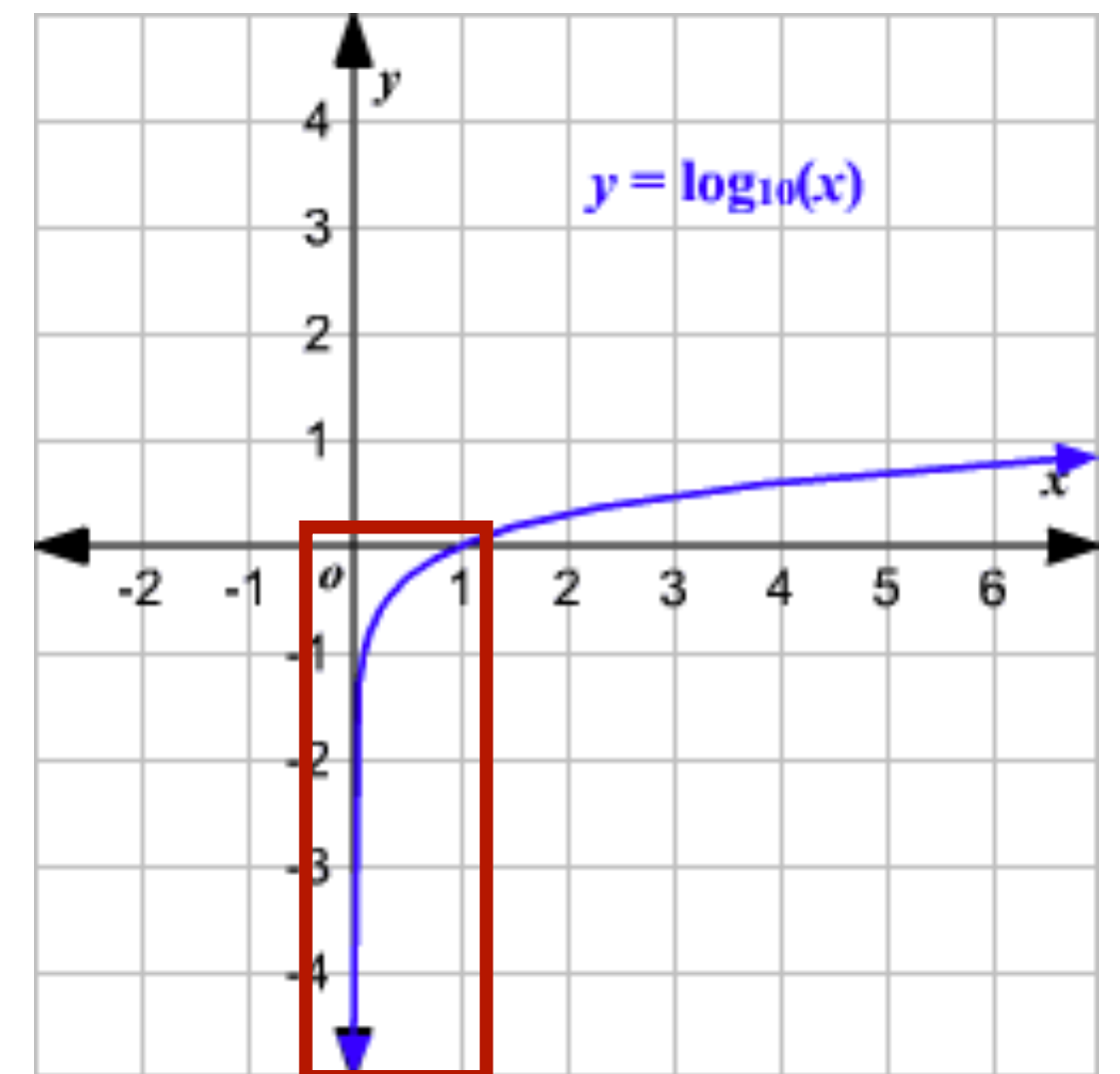


Scoring hypothesis in speech recognition

From acoustic signal to candidate transcriptions

Hypothesis	Score
the station signs are in deep in english	-14732
the stations signs are in deep in english	-14735
the station signs are in deep into english	-14739
the station 's signs are in deep in english	-14740
the station signs are in deep in the english	-14741
the station signs are indeed in english	-14757
the station 's signs are indeed in english	-14760
the station signs are indians in english	-14790
the station signs are indian in english	-14799
the stations signs are indians in english	-14807
the stations signs are indians and english	-14815

Why are these scores negative?



Scoring hypothesis in machine translation

From source language to target language candidates

Hypothesis	Score
we must also discuss a vision .	-29.63
we must also discuss on a vision .	-31.58
it is also discuss a vision .	-31.96
we must discuss on greater vision .	-36.09
⋮	⋮

How are language models used?

How is it incorporated in applications?

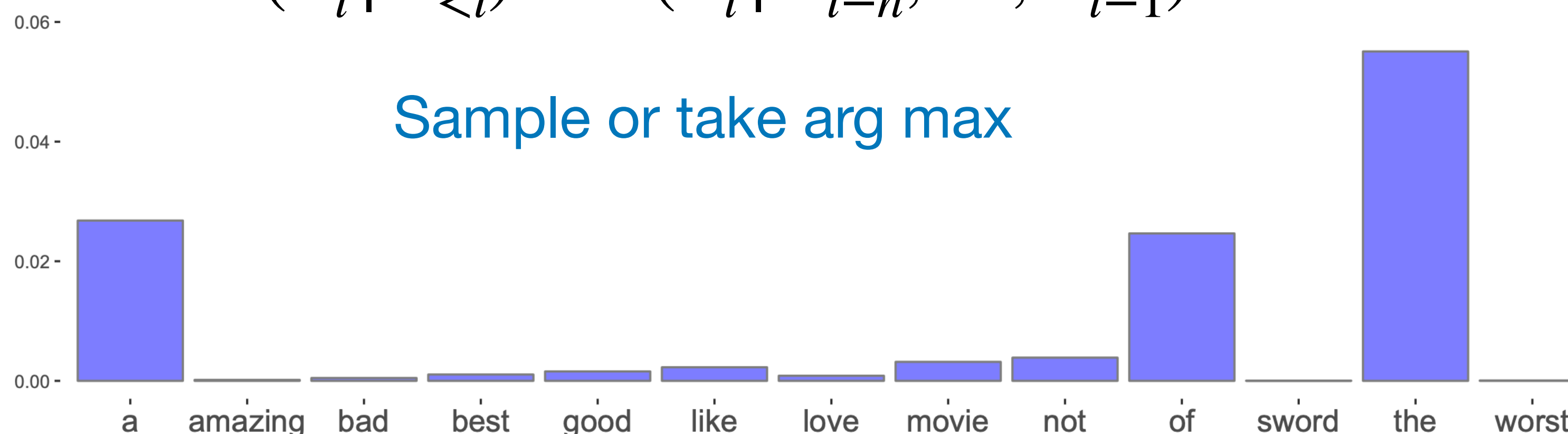
How do we use the language model for generating?

How do you generate text from an n-gram model?

- Generate tokens from left to right
- Use the last $i - 1$ words for context
- Select from multinomial over the vocabulary that include a **STOP** token. Repeat until **STOP** is generated.
- When generating word w_i given previous words $w_{<i}$

context1	context2	generated word
START	START	The
START	The	dog
The	dog	walked
dog	walked	in

$$P(w_i | w_{<i}) = P(w_i | w_{i-n}, \dots, w_{i-1})$$



(Credits: David Bamman)

Example generations

Unigram

release millions See ABC accurate President of Donald Will cheat them a CNN megynkelly experience @ these word out- the

Bigram

Thank you believe that @ ABC news, Mississippi tonight and the false editorial I think the great people Bill Clinton . "

Trigram

*We are going to MAKE AMERICA GREAT AGAIN!
#MakeAmericaGreatAgain <https://t.co/DjkdAzT3WV>*

Typical LMs are not sufficient to handle long-range dependencies

“**Alice/Bob** could not go to work that day because
she/he had a doctor’s appointment”

Example generation: Shakespeare

1
gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have
–Hill he late speaks; or! a more to leg less first you enter

2
gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.
–What means, sir. I confess she? then all sorts, he is trim, captain.

3
gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.
–This shall forbid it should be branded, if renown made it empty.

4
gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;
–It cannot be but so.

<https://web.stanford.edu/~jurafsky/slp3/3.pdf>

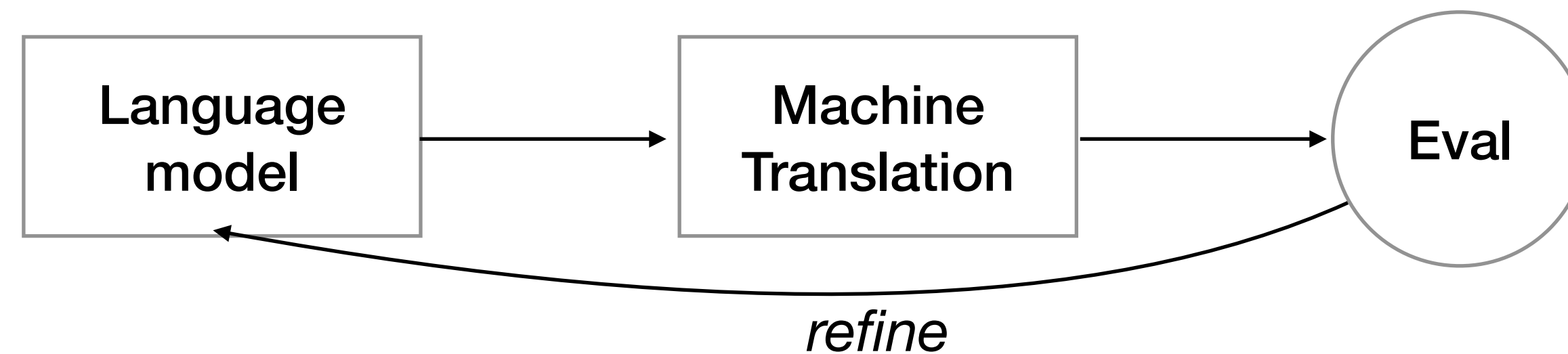
Evaluating Language Models

Evaluation

- **Extrinsic**: measure how useful the language model is at some task (MT, ASR, etc).
- **Intrinsic**: measure how good we are at modeling language

Extrinsic evaluation

- Train LM → apply to task → observe accuracy



- Directly optimized for downstream tasks
 - higher task accuracy → better model
- Expensive, time consuming
- Hard to optimize downstream objective (indirect feedback)

Evaluating language models

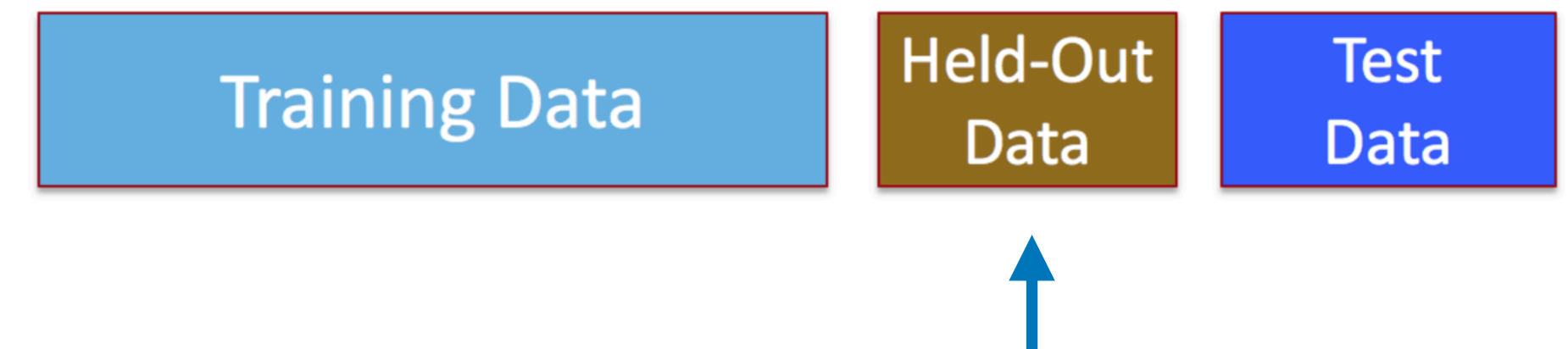
- A good language model should assign higher probability to typical, grammatically correct sentences

- Research process:

- **Train** parameters on a suitable training corpus
 - Assumption: observed sentences $\sim$ good sentences

- **Test** on *different, unseen* corpus
 - Training on any part of test set not acceptable!

- **Evaluation metric**



Use held-out data (aka validation set/development set) for hyperparameter tuning



Evaluation of language models

Computing the probability of the test corpus

- So far we've seen the probability of a single sentence:

$$P(s) = P(w_1, \dots, w_n)$$

- What is the probability of a collection of sentences for an unseen test corpus T ?
- Let $T = s_1, \dots, s_m$ be a text corpus with m sentences.
 - Remember T is assumed to be separate from the training data used to train our language model
- To compute the overall probability of the test corpus $P(T)$, we assume that the sentences are **independent**.

Evaluation of language models

Computing the probability of the test corpus

- Given a test corpus $T = s_1, \dots, s_m$ of independent sentences, the probability of $P(T)$ is:

$$P(T) = P(s_1, \dots, s_m) = P(s_1) \cdot P(s_2) \cdots P(s_m) = \prod_{i=1}^m P(s_i)$$

- A language model is better if the probability $P(T)$ of an unseen corpus is high. So we want to maximize $P(T)$.

What happens to $P(T)$ as m increases?

Evaluation of language models

Computing the **average** probability of the test corpus

- Given a test corpus $T = s_1, \dots, s_m$ of independent sentences, the probability of $P(T)$ is:

$$P(T) = \prod_{i=1}^m P(s_i) \quad \text{higher } P(T) = \text{better LM}$$

- But T can be any size and $P(T)$ will be lower if T is larger.
- So let's compute the **average** probability. Let M be the total number of tokens in the test corpus T .

$$M = \sum_{i=1}^m \text{length}(s_i)$$

- The average **log** probability of the test corpus T is:

$$\ell = \frac{1}{M} \log_2 \prod_{i=1}^m P(s_i) = \frac{1}{M} \sum_{i=1}^m \log_2 P(s_i)$$

Evaluation of language models

Perplexity

M = total # of words

m = # of sentences

- The average log probability of the test corpus T is:

$$\ell = \frac{1}{M} \sum_{i=1}^m \log_2 P(s_i)$$

higher ℓ = better LM

Note that ℓ is
a negative number

- Language models are evaluated using **perplexity**

$$\text{ppl}(T) = 2^{-\ell} \quad \text{lower ppl} = \text{better LM}$$

$\text{ppl}(T)$ is
a positive number

Note that the exponent $(-\ell)$ can be regarded as the **cross entropy** between the empirical distribution of test corpus and the language model

Intuition: **Measure of model's uncertainty about next word**

Evaluation of language models

Exercises on perplexity

Given a test corpus T of m sentences and M words,

1. What is the perplexity of the test corpus if we have an n -gram model with uniform probability, i.e. for vocabulary V , the n -gram probability is given by

$$P(w_i | w_{i-n}, \dots, w_{i-1}) = \frac{1}{|V|} \quad \forall w_i$$

2. Show that the perplexity $2^{-\ell} = 2^{-\frac{1}{M} \log_2 \prod_{i=1}^m P(s_i)} = \frac{1}{\sqrt[M]{\prod_{i=1}^m P(s_i)}}$
3. What happens to the perplexity if any of the n -gram probability for computing $P(T)$ is zero?

Pros and cons of perplexity

Pros	Cons

Pros and cons of perplexity

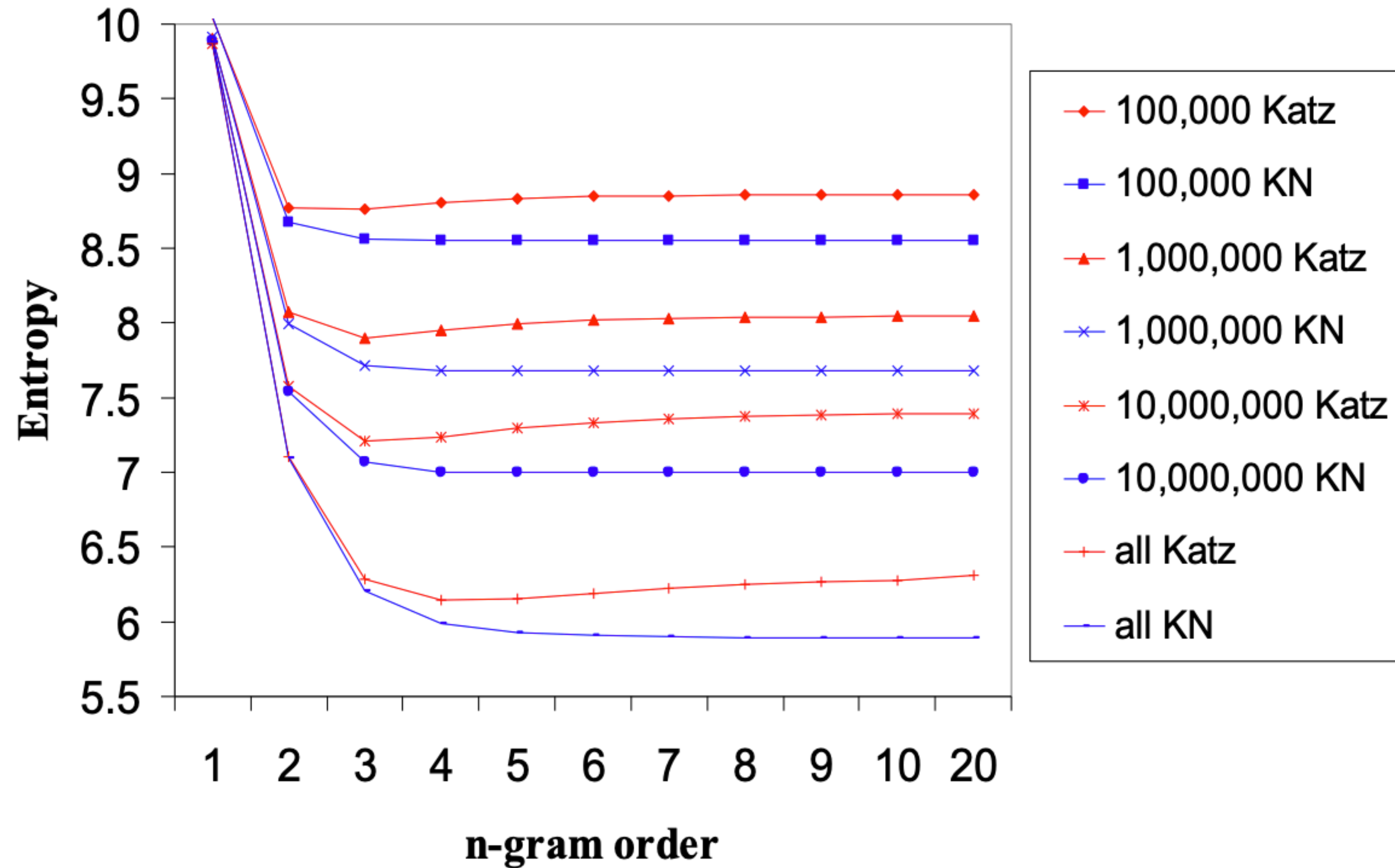
Pros	Cons
Easy to compute	Domain match between train and test
standardized	Limited to sequence models
directly useful, easy to use to correct sentences	might not correspond to end task optimization
nice theoretical interpretation - matching distributions	$\log 0$ undefined
	can be cheated by predicting common tokens
	size of test set matters
	can be sensitive to low prob tokens/ sentences

Typical values of Perplexity

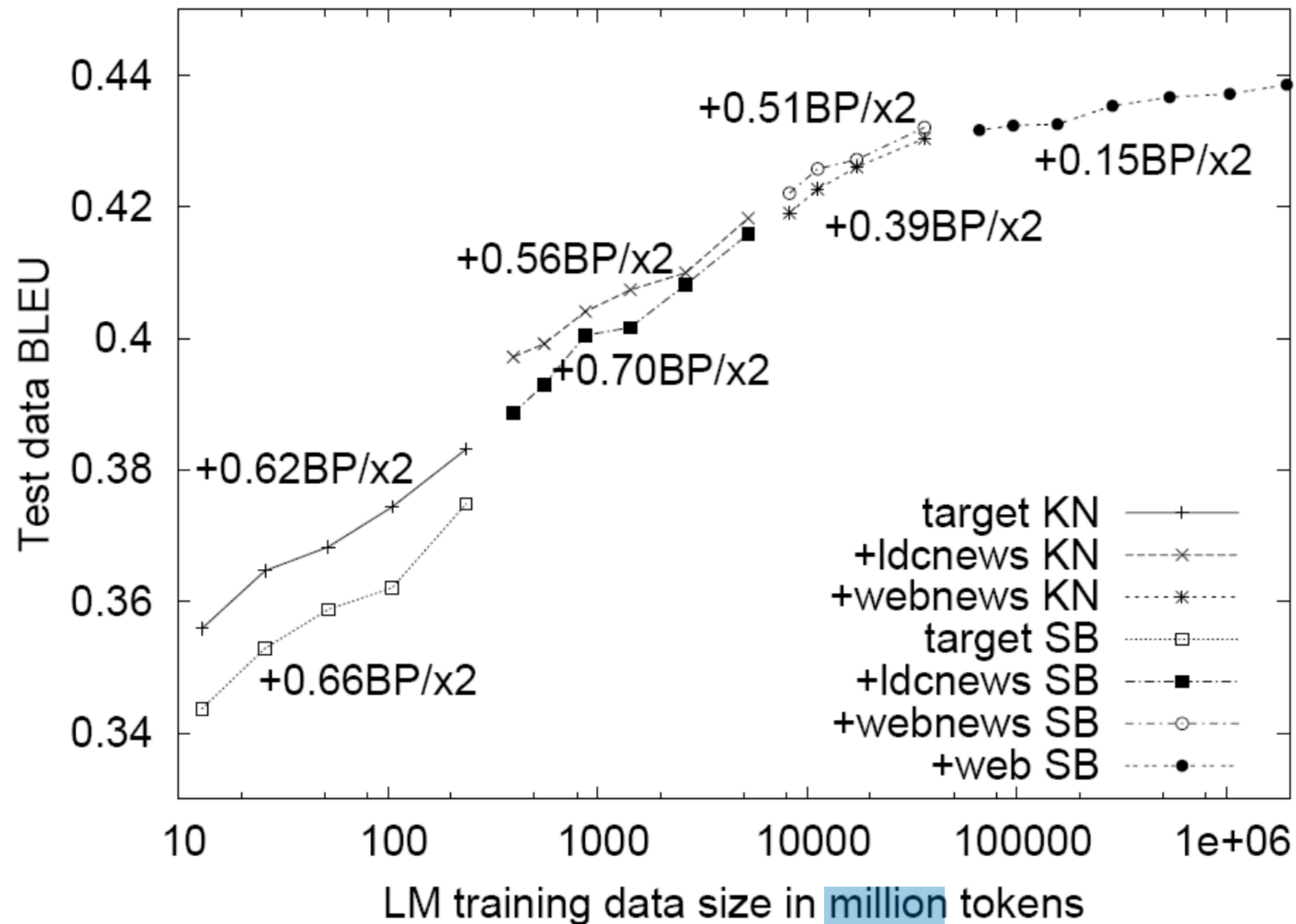
From 'One Billion Word Benchmark for Measuring Progress in Statistical Language Modeling' by Chelba+ (Google)

Model	Num. Params [billions]	Training Time		Perplexity
		[hours]	[CPUs]	
Interpolated KN 5-gram, 1.1B n-grams (KN)	1.76	3	100	67.6
Katz 5-gram, 1.1B n-grams	1.74	2	100	79.9
Stupid Backoff 5-gram (SBO)	1.13	0.4	200	87.9
Interpolated KN 5-gram, 15M n-grams	0.03	3	100	243.2
Katz 5-gram, 15M n-grams	0.03	2	100	127.5
Binary MaxEnt 5-gram (n-gram features)	1.13	1	5000	115.4
Binary MaxEnt 5-gram (n-gram + skip-1 features)	1.8	1.25	5000	107.1
Hierarchical Softmax MaxEnt 4-gram (HME)	6	3	1	101.3
Recurrent NN-256 + MaxEnt 9-gram	20	60	24	58.3
Recurrent NN-512 + MaxEnt 9-gram	20	120	24	54.5
Recurrent NN-1024 + MaxEnt 9-gram	20	240	24	51.3

More data is better!



More data is better!



KN = Kneser-Ney
SB = Stupid Backoff

1 Billion tokens

Where are we now?

Neural models with lots of data!

300 Billion tokens

Training data: mix of web + books + Wikipedia

Model Name	n_{params}
GPT-3 Small	125M
GPT-3 Medium	350M
GPT-3 Large	760M
GPT-3 XL	1.3B
GPT-3 2.7B	2.7B
GPT-3 6.7B	6.7B
GPT-3 13B	13.0B
GPT-3 175B or “GPT-3”	175.0B

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Open AI's GPT 3

Language Models are Few-Shot Learners

(Brown et al, 2020)

<https://arxiv.org/pdf/2005.14165.pdf>